

■ Live Customer Support Chat

Full Phased Project Structure

Django · Channels · WebSockets · Redis · PostgreSQL

6 Phases · ~14 Weeks · Production-Ready

6
Project Phases

14
Estimated Weeks

3
User Roles

12+
Core Features

Project Overview

This document outlines the complete phased development plan for a real-time Live Customer Support Chat system built with Django and WebSockets. The system enables businesses to provide instant customer support through a live chat interface, with agents managing multiple conversations from a centralized dashboard. The project is structured across six sequential phases covering environment setup, core WebSocket mechanics, full feature development, security hardening, testing, and production deployment.

Phase Summary

#	Phase	Duration	Key Output
1	Project Setup & Architecture	Week 1–2	Dev environment, models, routing
2	Core WebSocket Engine	Week 3–4	Real-time messaging working
3	Agent Dashboard & Queue Logic	Week 5–7	Full agent UI, queue system
4	Advanced Features	Week 8–10	Typing indicators, history, ratings
5	Security, Auth & Performance	Week 11–12	JWT auth, rate limiting, Redis tuning

6	Testing & Deployment	Week 13–14	Docker, CI/CD, production launch
---	----------------------	------------	----------------------------------

Goals

- Set up a clean Django + Channels development environment
- Define the full database schema for chat sessions and messages
- Configure Redis as the channel layer backend
- Establish project folder structure and coding conventions

Folder Structure

```
support_chat/  
├── config/  
│   ├── settings.py  
│   ├── asgi.py  
│   └── urls.py  
├── chat/  
│   ├── consumers.py  
│   ├── routing.py  
│   ├── models.py  
│   ├── views.py  
│   ├── serializers.py  
│   └── templates/  
├── agents/  
│   ├── models.py  
│   └── views.py  
├── static/  
├── requirements.txt  
└── docker-compose.yml
```

Database Models

ChatSession	Stores each conversation: customer name, agent FK, status (waiting/active/closed), timestamp
Message	Individual messages: FK to session, sender type (customer/agent), content, timestamp, is_read
Agent	Extends Django User: availability status, department, active_chats count, rating average
Rating	Post-chat rating: FK to session, score 1–5, optional feedback text

Tech Stack Setup

Package	Version	Purpose
Django	4.2+	Core web framework
channels	4.x	WebSocket support via ASGI
channels-redis	4.x	Redis channel layer backend
djangorestframework	3.x	REST API for HTTP endpoints
daphne	4.x	ASGI server for production
psycopg2	2.x	PostgreSQL adapter

Deliverables

- Working Django project with channels installed
- All models created and migrated
- Redis running via Docker
- Basic ASGI configuration in asgi.py

Goals

- Build the ChatConsumer handling connect, disconnect, and receive
- Implement group-based message broadcasting via Redis channel layer
- Create basic customer-facing chat widget (HTML + vanilla JS)
- Messages saved to DB on every send event

WebSocket Flow

['Step', 'Actor', 'Action', 'Channel Layer Event']			
1	Customer	Opens chat page → WS connect()	Join group: chat_{session_id}
2	Customer	Types & sends message	group_send → chat_message
3	Consumer	receive() fires	Saves to DB, broadcasts to group
4	Agent	Receives message instantly	chat_message handler → send()
5	Agent	Sends reply	Same broadcast flow in reverse

Consumer Events Handled

connect()	Accept WS, add to group, mark session active
disconnect()	Remove from group, optionally mark session closed
receive() – message	Validate, save Message to DB, broadcast to group
receive() – typing	Broadcast typing event (not saved to DB)
receive() – read	Mark messages as read, broadcast read-receipt
chat_message()	Push message JSON to the client WebSocket
typing_indicator()	Push typing event to the other party

Frontend WebSocket Client

```
const ws = new WebSocket(`ws://HOST/ws/chat/${sessionId}/`);

ws.onopen = () => console.log('Connected');
ws.onmessage = (e) => handleEvent(JSON.parse(e.data));
ws.onclose = () => attemptReconnect();

function sendMessage(text) {
  ws.send(JSON.stringify({ type: 'message', content: text }));
}
```

```
}  
function sendTyping() {  
  ws.send(JSON.stringify({ type: 'typing' }));  
}
```

Deliverables

- ChatConsumer fully implemented and tested
- Messages persist to PostgreSQL
- Customer widget sends and receives messages
- Typing indicator working end-to-end

Goals

- Build the agent-facing dashboard with live chat queue
- Implement chat claiming — agents pick up waiting conversations
- Show agent availability status (online / busy / offline)
- Notify all online agents when a new customer connects
- Limit agents to a configurable max concurrent chats

Agent Dashboard Features

Feature	Description	WebSocket Event
Live Queue Panel	Shows all waiting chats in real-time	new_chat_waiting
Active Chats Sidebar	Lists claimed conversations	chat_claimed
Chat Window	Full message thread with the customer	chat_message
Typing Badge	Shows when customer is typing	typing_indicator
Agent Status Toggle	Online / Busy / Offline switcher	agent_status_change
Unread Count Badge	Highlights chats with unread messages	message_read
Transfer Chat	Hand off conversation to another agent	chat_transferred

Queue Distribution Logic

New chat arrives	Broadcast new_chat_waiting to all online agents group
Agent claims chat	Atomic DB update sets agent FK, status → active
Claim conflict	First write wins; losers receive already_claimed event
No agents online	Session stays in queue, customer sees estimated wait time
Max load reached	Agent auto-toggled to Busy, removed from eligible pool
Agent disconnects	Unclaim active chats, re-broadcast to queue

New Consumer: AgentConsumer

```
class AgentConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        # Add to agents_online group
        await self.channel_layer.group_add('agents_online', self.channel_name)
        await self.accept()

    async def receive(self, text_data):
        data = json.loads(text_data)
        if data['type'] == 'claim_chat':
            await self.claim_session(data['session_id'])
        elif data['type'] == 'status_change':
            await self.update_agent_status(data['status'])
```

Deliverables

- Agent dashboard UI complete
- Real-time queue broadcasts working
- Chat claiming with conflict resolution
- Agent status management

Features Built in This Phase

Chat History & Pagination

On reconnect, load last N messages from DB. Infinite scroll upward loads older messages via REST API. Customer sees full thread even after refresh.

File & Image Attachments

Customer or agent uploads image/file via AJAX. URL stored in Message model. WebSocket event notifies other party to render the attachment inline.

Post-Chat Rating

After session closes, customer receives a star-rating prompt. Score + optional text saved to Rating model. Agents see their average on the dashboard.

Canned Responses

Agents store common replies. Searchable dropdown auto-fills the chat input. Reduces response time for repetitive queries.

Read Receipts

Messages get an `is_read` flag. When the other party views a message, a read event fires and ticks appear (WhatsApp-style double tick).

Chat Transfer

Agent selects a target agent and transfers the session. Both agents notified, customer sees 'You have been transferred to [Agent Name]'.

Reconnection Handling

Client JS implements exponential back-off reconnect. On reconnect, fetches missed messages via REST and re-joins the WebSocket group.

REST API Endpoints (Django REST Framework)

Method	Endpoint	Purpose
POST	<code>/api/sessions/</code>	Create a new chat session
GET	<code>/api/sessions/{id}/messages/</code>	Paginated message history
POST	<code>/api/sessions/{id}/close/</code>	Close and archive session
POST	<code>/api/sessions/{id}/rate/</code>	Submit post-chat rating
GET	<code>/api/agents/</code>	List available agents
POST	<code>/api/attachments/</code>	Upload file/image attachment

Deliverables

- Chat history loads on reconnect
- File upload & inline preview
- Star rating flow complete
- Read receipts working
- All REST API endpoints documented

Authentication

Customer Auth	Anonymous session token issued on chat start; stored in cookie/localStorage; passed in WS
Agent Auth	JWT issued on login; sent in WS handshake header; verified in ChatConsumer.connect()
Token Refresh	Short-lived access tokens (15 min) + long refresh tokens; silent refresh in JS before expiry
WS Auth Guard	Custom middleware validates token in scope before consumer runs; reject unauthenticated c

Security Hardening

- CSRF protection on all HTTP endpoints
- Input sanitization — strip HTML/scripts from all message content
- Rate limiting — max N messages per second per session (Django Ratelimit or custom Redis counter)
- XSS prevention — escape all content before rendering in the browser
- CORS configured to whitelist only your frontend domain
- WebSocket origin check — reject connections from unknown origins
- File upload validation — whitelist MIME types, scan for malware with ClamAV

Performance Optimizations

Optimization	How	Impact
DB query reduction	select_related() on Message queryset	Fewer SQL joins
Message caching	Cache last 50 messages per session in Redis	Fast history load
Channel layer tuning	Increase Redis connection pool size	Higher concurrency
Async consumers	Use AsyncWebsocketConsumer throughout	Non-blocking I/O
Index DB columns	Index session_id, timestamp on Message table	Fast pagination
WS compression	Enable permessage-deflate on Daphne/Nginx	~60% less bandwidth

Deliverables

- JWT-based agent authentication
- Secure customer session tokens
- Rate limiting active on message events
- All inputs sanitized
- Performance benchmarks documented

Testing Strategy

Test Type	Tool	What It Covers
Unit Tests	pytest-django	Models, serializers, helper functions
Consumer Tests	channels.testing	WebSocket connect/disconnect/receive logic
Integration Tests	pytest + WebsocketCommunicator	Full send → broadcast → receive flow
API Tests	DRF APIClient	All REST endpoints, auth, pagination
Load Testing	Locust	1000 concurrent WS connections
E2E Tests	Playwright	Full browser customer + agent flow

Docker Compose Setup

```
services:
  web:
    build: .
    command: daphne config.asgi:application
    ports: ['8000:8000']
    depends_on: [db, redis]

  db:
    image: postgres:15

  redis:
    image: redis:7-alpine

  nginx:
    image: nginx:alpine
    # Proxies HTTP + WebSocket upgrades
```

Nginx WebSocket Config

```
location /ws/ {
  proxy_pass http://web:8000;
  proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade;
  proxy_set_header Connection 'upgrade';
  proxy_set_header Host $host;
}
```

CI/CD Pipeline (GitHub Actions)

On push to main	Run lint (flake8) + full test suite
On PR	Run tests + post coverage report as PR comment
On tag/release	Build Docker image, push to registry, deploy to server

Deployment Checklist

- DEBUG=False, SECRET_KEY from environment variable
- ALLOWED_HOSTS set to production domain
- HTTPS + WSS enforced via Nginx SSL termination
- Database backups scheduled (pg_dump + S3)
- Redis persistence enabled (AOF mode)
- Sentry connected for error monitoring
- Uptime monitoring (e.g. Better Uptime) configured

Final Deliverables

- Full test suite with >80% coverage
- Docker Compose + Dockerfile
- Nginx config with WS support
- CI/CD pipeline running
- Production deployment live
- README with setup and architecture docs

Project Completion Summary

Phase	Focus	Weeks	Status Gate
1 – Setup	Models, Redis, project scaffold	1–2	All models migrated, Redis connected
2 – WS Engine	ChatConsumer, broadcast, widget	3–4	Messages flow end-to-end
3 – Dashboard	Agent UI, queue, claiming	5–7	Agents can claim & reply
4 – Features	History, files, ratings, receipts	8–10	All features demoed
5 – Security	JWT, rate limits, sanitization	11–12	Pen test checklist passed
6 – Deploy	Tests, Docker, CI/CD, production	13–14	App live & monitored

Skills You Will Demonstrate

• Django Channels (ASGI) • WebSocket protocol • Redis channel layers • Async Python consumers	• JWT authentication • REST API design (DRF) • PostgreSQL & query optimization • Docker & Nginx	• pytest + Playwright testing • CI/CD with GitHub Actions • Security hardening • Real-time UX patterns
---	---	--